

Requirements Document: Sacct Efficiency & Usage Reporter

1. Overview

A pure-Ruby command-line tool that queries SLURM accounting data via `sacct`, parses the results, and produces concise reports focused on job efficiency, queue times, and job outcomes. Intended as a 1–2 day project for a junior developer in an HPC environment.

2. Goals

- Convert raw `sacct` output into clear, aggregated summaries.
- Concentrate on the three highest-value areas: resource efficiency, queue time, and job outcomes.
- Remain simple, self-contained, and usable by a normal user on a SLURM cluster.

3. Scope

- Fetching data with `sacct`.
- Computing efficiency, queue-time, and outcome metrics.
- Grouping results overall, by partition, and by user.
- Basic filtering (date range, user, partition, state).
- Table output and optional CSV export.
- Basic error handling.

4. Functional Requirements

FR1 – Data retrieval

- Invoke `sacct` with appropriate flags.
- Support filters: start/end date, user, partition, job state.
- Handle empty result sets cleanly.

FR2 – Metrics

The tool must compute and report:

Category	Metrics	Grouping
Efficiency	CPU efficiency (mean, median); Memory efficiency (mean, median)	Overall, by partition, by user
Queue time	Median and 95th percentile	Overall, by partition, by user
Outcomes	% COMPLETED / FAILED / TIMEOUT / CANCELLED; basic exit-code summary	Overall, by partition, by user

FR3 – Output

- Default: clean aligned tables to stdout.
- Optional: CSV export of the aggregated data.
- Clear units (% , hours, counts).

FR4 – Error handling

- Detect missing sacct binary.
- Handle permission errors and empty results with clear messages.
- Non-zero exit status on failure.

5. CLI Interface (sketch)

sacct-report [options]

Options:

--start DATE	Start of period (YYYY-MM-DD)
--end DATE	End of period (YYYY-MM-DD)
--user USER	Filter by user (default: current user)
--partition NAME	Filter by partition
--state LIST	Comma-separated states (optional filter)
--csv FILE	Write aggregated data to CSV
--help	

6. Data Source & Assumptions

- Primary source: sacct --parsable2 (or --json if reliable).
- Key fields: JobID, User, Partition, State, Submit, Start, End, Elapsed, AllocCPUS, TotalCPU, ReqMem, MaxRSS, ExitCode.
- Queue time = Start – Submit.
- CPU efficiency = TotalCPU / (Elapsed × AllocCPUS).
- Memory efficiency = MaxRSS / ReqMem (with safe handling of missing/zero values).
- Runs with the privileges of the invoking user.

7. Success Criteria

- Correctly calculates the metrics in FR2.
- Supports grouping by overall / partition / user.
- Produces readable output.
- Handles the main error cases without crashing.
- Completable and usable within 1–2 days of junior developer effort.